UNIFECAF

EDUARDO SOUZA MATTOS

SISTEMA AUTOMOTIVO: GESTÃO DE ESTOQUE DE VEÍCULOS

Relatório da parte teórica

São Paulo
2026

EDUARDO SOUZA MATTOS – R.A. 35984

SISTEMA AUTOMOTIVO: GESTÃO DE ESTOQUE DE VEÍCULOS

Relatório da parte teórica

Relatório da parte teórica apresentado à disciplina Object Oriented Programming, do curso de Análise e Desenvolvimento de Sistemas da UNIFECAF, como requisito parcial de avaliação.

São Paulo

2026

RESUMO

Este relatório apresenta a parte teórica do trabalho da disciplina Object Oriented Programming: o projeto de um sistema de gestão de estoque de veículos para uma concessionária. O problema tratado é a desorganização dos dados de estoque, que dificulta consultas e atualizações e prejudica vendedores, clientes e gestores. A solução proposta é um sistema CRUD (cadastro, consulta, atualização e remoção) orientado a objetos, desenvolvido em Java, com Spring Boot, Spring Data JPA, API REST, banco de dados MySQL e uma interface web simples. O texto descreve o levantamento de requisitos, os requisitos funcionais e não funcionais, a modelagem das entidades Marca, Modelo e Veículo, a aplicação dos conceitos de encapsulamento, construtores, herança, composição, polimorfismo, abstração e tratamento de exceções, a arquitetura em camadas e a justificativa das tecnologias escolhidas. Todo o código-fonte utilizado no trabalho é apresentado nos apêndices. Conclui-se que a orientação a objetos, combinada com a arquitetura em camadas, produz um código organizado, de fácil manutenção e adequado ao problema proposto.

Palavras-chave: Programação orientada a objetos. Java. Spring Boot. API REST. Gestão de estoque.

ABSTRACT

This report presents the theoretical part of the Object Oriented Programming course project: the design of a vehicle inventory management system for a car dealership. The problem addressed is the disorganization of inventory data, which makes searching and updating records difficult and harms salespeople, customers and managers. The proposed solution is an object-oriented CRUD system (create, read, update and delete) developed in Java with Spring Boot, Spring Data JPA, a REST API, a MySQL database and a simple web interface. The text describes the requirements elicitation, the functional and non-functional requirements, the modeling of the Brand, Model and Vehicle entities, the application of encapsulation, constructors, inheritance, composition, polymorphism, abstraction and exception handling, the layered architecture and the justification of the chosen technologies. All source code used in the project is presented in the appendices. The report concludes that object orientation combined with a layered architecture yields organized, maintainable code that is suitable for the proposed problem.

Keywords: Object-oriented programming. Java. Spring Boot. REST API. Inventory management.

LISTA DE FIGURAS

Figura 1 – Diagrama de classes do sistema 14

LISTA DE QUADROS

Quadro 1 – Frentes da solução, com prós e contras 9
Quadro 2 – Perguntas do levantamento de requisitos e respostas simuladas 11
Quadro 3 – Requisitos funcionais 12
Quadro 4 – Requisitos não funcionais 12
Quadro 5 – Entidades e enumerações do sistema 13
Quadro 6 – Modelo físico das tabelas 14
Quadro 7 – Camadas da aplicação 17
Quadro 8 – Operações da API REST sobre veículos 17
Quadro 9 – Tecnologias utilizadas e justificativas 19

SUMÁRIO

1 INTRODUÇÃO 9
1.1 Contextualização do problema 9
1.2 Objetivos 9
1.3 Organização do trabalho 10
2 LEVANTAMENTO DE REQUISITOS 11
3 REQUISITOS DO SISTEMA 12
3.1 Requisitos funcionais 12
3.2 Requisitos não funcionais 12
3.3 Regras de negócio 12
4 MODELAGEM DO DOMÍNIO 13
4.1 Modelo físico 14
5 APLICAÇÃO DOS CONCEITOS DE ORIENTAÇÃO A OBJETOS 15
5.1 Classes, objetos, atributos e métodos 15
5.2 Encapsulamento 15
5.3 Construtores e sobrecarga 15
5.4 Herança e composição 16
5.5 Abstração e polimorfismo 16
5.6 Enumerações 16
5.7 Tratamento de exceções 16
6 ARQUITETURA EM CAMADAS E API REST 17
6.1 Camadas 17
6.2 API REST 17
6.3 Consulta com filtros opcionais 18
6.4 Interface web 18
7 JUSTIFICATIVA DAS TECNOLOGIAS ESCOLHIDAS 19
8 CONSIDERAÇÕES FINAIS 20
REFERÊNCIAS 21
APÊNDICE A – ARQUIVO DE DEPENDÊNCIAS DO PROJETO (POM.XML) . . . 23
APÊNDICE B – CONFIGURAÇÃO DO BANCO DE DADOS E COMANDOS DE EXECUÇÃO 24
APÊNDICE C – CLASSE PRINCIPAL E ENUMERAÇÕES 25
APÊNDICE D – CAMADA MODEL: ENTIDADES 26

APÊNDICE E – CAMADA REPOSITORY: ACESSO AO BANCO DE DADOS . . . 31
APÊNDICE F – CAMADA SERVICE: REGRAS DE NEGÓCIO 32
APÊNDICE G – CAMADA CONTROLLER E TRATAMENTO DE EXCEÇÕES . . 36
APÊNDICE H – INTERFACE WEB (INDEX.HTML) 42
APÊNDICE I – MODELO FÍSICO DO BANCO DE DADOS (SCRIPT SQL) 49

1 INTRODUÇÃO

1.1 Contextualização do problema

Concessionárias e revendedoras de veículos lidam diariamente com grande volume de informações: marca, modelo, ano, cor, preço, quilometragem e situação de cada unidade. Quando esses dados ficam em planilhas soltas, cadernos ou sistemas fragmentados, o estoque se torna difícil de consultar e de manter atualizado. A principal causa do problema é a falta de organização e de um local único e confiável para os dados do estoque.

Três grupos são afetados. Os vendedores oferecem ao cliente veículos já vendidos ou com preço desatualizado, perdendo tempo e credibilidade. Os clientes não conseguem filtrar o que procuram e recebem informações confusas, o que pode levá-los a decidir mal ou a desistir da compra. Os gerentes e administradores têm dificuldade para acompanhar a disponibilidade, controlar o inventário e gerar relatórios, o que prejudica decisões estratégicas.

As barreiras para resolver o problema são a pouca familiaridade da equipe com as tecnologias envolvidas, o desconhecimento de conceitos de banco de dados e a dificuldade de integração entre as partes do sistema. Sem uma solução, persistem a dificuldade de gestão e o risco de vendas perdidas, de retrabalho e de decisões baseadas em dados incorretos.

A solução proposta é um sistema CRUD (cadastrar, consultar, atualizar e remover) orientado a objetos, com API REST em Java e Spring Boot, persistência em MySQL e uma interface web simples. O Quadro 1 resume as três frentes consideradas, com seus prós e contras.

Quadro 1 – Frentes da solução, com prós e contras

Frente	**Prós**	**Contras**
Solução com Spring Boot	Configuração rápida, servidor embutido, grande ecossistema e comunidade.	Curva de aprendizado para iniciantes; muitas convenções a conhecer.
Comunicação com banco de dados (MySQL e JPA)	Dados persistentes e consultáveis; menos código SQL manual.	Exige conhecimento de modelagem e de configuração da conexão.
Integração com front-end	Interface amigável para vendedores e clientes.	Aumenta o esforço de desenvolvimento e de integração.

Fonte: Elaborado pelo autor (2026).

1.2 Objetivos

O objetivo geral é projetar e implementar um sistema de gestão de estoque de veículos que aplique os conceitos de programação orientada a objetos estudados na disciplina (UNIFECAF, [s.d.b], [s.d.c], [s.d.d]). Os objetivos específicos são: levantar os requisitos do sistema; modelar as entidades do domínio; aplicar encapsulamento, herança, composição e polimorfismo; organizar o código em camadas; persistir os dados em banco relacional; e disponibilizar uma interface web para uso das operações.

1.3 Organização do trabalho

A Seção 2 apresenta o levantamento de requisitos e a Seção 3 os requisitos do sistema. A Seção 4 descreve a modelagem do domínio, e a Seção 5, a aplicação dos conceitos de orientação a objetos. A Seção 6 trata da arquitetura em camadas e da API REST, e a Seção 7 justifica as tecnologias escolhidas. A Seção 8 reúne as considerações finais. Todo o código-fonte do trabalho está reproduzido nos Apêndices A a I, e a execução do sistema, com as evidências, é apresentada no relatório da parte prática.

Este documento segue as normas da Associação Brasileira de Normas Técnicas para trabalhos acadêmicos, citações, referências, sumário, resumo e numeração progressiva (ABNT NBR 14724:2024, NBR 10520:2023, NBR 6023:2018, NBR 6027:2012, NBR 6028:2021 e NBR 6024:2012).

2 LEVANTAMENTO DE REQUISITOS

O levantamento de requisitos foi conduzido por meio de perguntas a um responsável fictício de uma concessionária, em conformidade com o cenário proposto no enunciado do trabalho (UNIFECAF, [s.d.a]). As respostas são simuladas e servem de base para os requisitos da Seção 3. O Quadro 2 reúne as doze perguntas e as respectivas respostas.

Quadro 2 – Perguntas do levantamento de requisitos e respostas simuladas

Nº	**Pergunta**	**Resposta simulada**
1	Quais informações precisam ser registradas de cada veículo?	Marca, modelo, ano de fabricação, cor, preço, quilometragem e situação (disponível, reservado ou vendido).
2	Como os dados do estoque são controlados hoje?	Por planilhas separadas por loja, o que gera versões diferentes e informações desencontradas.
3	Quem utilizará o sistema?	Vendedores (consulta e atualização), gerentes (acompanhamento do estoque) e administradores (cadastros em geral).
4	Marcas e modelos devem ter cadastro próprio?	Sim. Cada marca possui vários modelos, e cada veículo deve estar ligado a um modelo, evitando nomes digitados de formas diferentes.
5	Por quais critérios a equipe busca veículos?	Marca, modelo, faixa de preço, ano e situação (por exemplo, apenas os disponíveis).
6	Quais dados costumam mudar após o cadastro?	Preço (promoções), quilometragem (veículos seminovos) e situação de disponibilidade.
7	O que deve acontecer quando um veículo é vendido ou descontinuado?	Pode ser removido do estoque ou ter a situação alterada para vendido, conforme a decisão do gerente.
8	Existem veículos de tipos diferentes (carro, moto, caminhão)?	Sim, e o tipo deve poder ser usado como filtro de busca.
9	Qual é o volume aproximado de veículos em estoque?	Algumas centenas por loja, com consultas frequentes durante o atendimento.
10	Qual é o tempo de resposta aceitável nas consultas?	Poucos segundos, pois o cliente costuma estar presente no atendimento.
11	Que cuidados são necessários com a qualidade dos dados?	Não aceitar preço negativo, ano impossível ou quilometragem negativa.
12	O sistema precisa de interface gráfica desde o início?	Idealmente sim, mas a prioridade é a base funcionando; uma tela simples de listagem e filtro já atende.

Fonte: Elaborado pelo autor (2026), com respostas simuladas de um responsável fictício de concessionária.

As respostas mostram três necessidades centrais: manter um cadastro único e confiável, consultar o estoque por critérios variados e garantir a qualidade dos dados gravados. Mostram também que marcas e modelos precisam de cadastro próprio, para evitar divergências de digitação, e que o tipo do veículo deve ser um valor controlado.

3 REQUISITOS DO SISTEMA

3.1 Requisitos funcionais

Os requisitos funcionais descrevem o que o sistema deve fazer (Quadro 3).

Quadro 3 – Requisitos funcionais

Código	**Requisito**	**Descrição**
RF01	Cadastro de veículos	Registrar modelo, marca, ano de fabricação, cor, preço, quilometragem e situação.
RF02	Cadastro de marcas e modelos	Registrar marcas e modelos e associá-los aos veículos do estoque.
RF03	Consulta e filtro	Consultar veículos por marca, modelo, preço, ano, situação e tipo.
RF04	Atualização	Alterar preço, quilometragem e situação de um veículo já cadastrado.
RF05	Remoção	Remover veículos do estoque quando vendidos ou descontinuados.

Fonte: Elaborado pelo autor (2026).

3.2 Requisitos não funcionais

Os requisitos não funcionais descrevem qualidades que o sistema deve apresentar (Quadro 4).

Quadro 4 – Requisitos não funcionais

Código	**Requisito**	**Descrição**
RNF01	Usabilidade	Interface intuitiva, com navegação simples e clara para cadastro, consulta e atualização.
RNF02	Desempenho	Consultas e atualizações rápidas, inclusive com filtros por preço, ano e modelo.
RNF03	Integridade dos dados	Validação das informações antes de gravá-las (preço, ano e quilometragem válidos).
RNF04	Manutenibilidade	Código organizado em camadas, facilitando correções e novas funcionalidades.

Fonte: Elaborado pelo autor (2026).

3.3 Regras de negócio

A partir da pergunta 11 do levantamento, foram definidas regras de validação, implementadas nos métodos de acesso da classe Veiculo e na camada de serviço (Apêndices D e E): o preço deve ser maior que zero; a quilometragem não pode ser negativa; o ano de fabricação deve estar entre 1900 e o ano seguinte ao atual; e, na consulta, o preço mínimo não pode ser maior que o máximo. Quando uma regra é violada, o sistema recusa a operação e devolve uma mensagem de erro ao usuário.

4 MODELAGEM DO DOMÍNIO

O domínio do problema foi modelado em três entidades principais, Marca, Modelo e Veículo, mais duas enumerações para valores fixos, como resume o Quadro 5. Uma marca possui vários modelos e um modelo possui vários veículos. A marca de um veículo é obtida por meio do seu modelo, o que evita dados duplicados e inconsistentes.

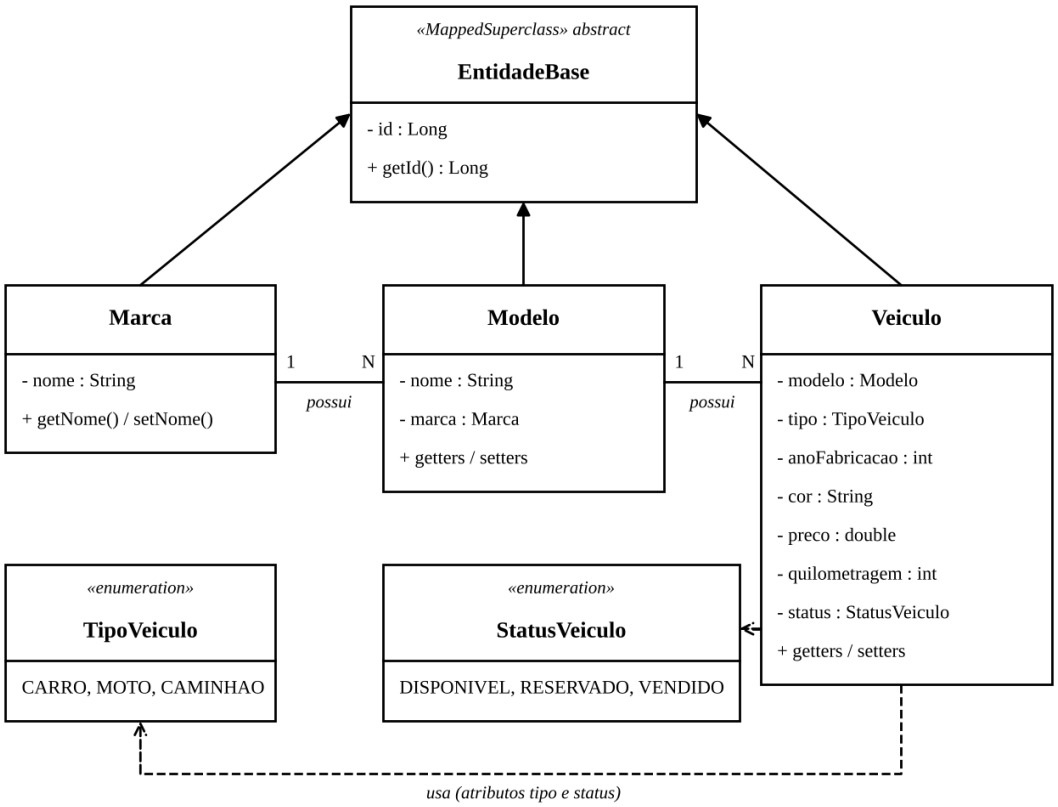
Quadro 5 – Entidades e enumerações do sistema

Elemento	**Atributos**	**Responsabilidade**
Marca	id, nome	Representa o fabricante (por exemplo, Toyota). Uma marca possui vários modelos.
Modelo	id, nome, marca	Representa o modelo comercial e pertence a uma marca. Um modelo pode ter vários veículos.
Veiculo	id, modelo, tipo, anoFabricacao, cor, preco, quilometragem, status	Unidade física em estoque; é a entidade central do sistema.
TipoVeiculo (enumeração)	CARRO, MOTO, CAMINHAO	Classifica o veículo e permite filtro por tipo.
StatusVeiculo (enumeração)	DISPONIVEL, RESERVADO, VENDIDO	Indica a situação de disponibilidade.

Fonte: Elaborado pelo autor (2026).

A Figura 1 representa as classes, a herança de EntidadeBase, as associações entre as entidades e o uso das enumerações.

Figura 1 – Diagrama de classes do sistema



Fonte: Elaborado pelo autor (2026).

4.1 Modelo físico

O modelo físico foi definido para o MySQL e corresponde ao esquema que a aplicação cria automaticamente a partir das classes. O Quadro 6 resume as tabelas, as colunas, as chaves e as restrições; o script completo (Apêndice I) cria o banco com as chaves primárias, as chaves estrangeiras, as restrições de validação (CHECK) e índices de apoio aos filtros de consulta (ORACLE, [s.d.]).

Quadro 6 – Modelo físico das tabelas

Tabela	**Colunas e tipos**	**Chaves e restrições**
marca	id BIGINT (autoincremento); nome VARCHAR(60)	PK (id); nome NOT NULL e UNIQUE
modelo	id BIGINT (autoincremento); nome VARCHAR(60); marca_id BIGINT	PK (id); FK marca_id para marca(id); todas as colunas NOT NULL
veiculo	id BIGINT (autoincremento); modelo_id BIGINT; tipo ENUM; ano_fabricacao INT; cor VARCHAR(30); preco DOUBLE; quilometragem INT; status ENUM	PK (id); FK modelo_id para modelo(id); NOT NULL em todas as colunas; CHECK ano_fabricacao >= 1900, preco > 0 e quilometragem >= 0

Fonte: Elaborado pelo autor (2026).

5 APLICAÇÃO DOS CONCEITOS DE ORIENTAÇÃO A OBJETOS

Os conceitos de orientação a objetos estudados na disciplina (UNIFECAF, [s.d.b], [s.d.c], [s.d.d]) foram aplicados diretamente no código. Esta seção descreve cada conceito, conforme a literatura (DEITEL; DEITEL, 2017; TURINI, 2014), e indica onde ele aparece nos apêndices.

5.1 Classes, objetos, atributos e métodos

Uma classe define o molde de um tipo de objeto, com atributos, que guardam o estado, e métodos, que descrevem o comportamento (DEITEL; DEITEL, 2017). Cada entidade do sistema é uma classe Java e cada veículo cadastrado é um objeto independente, criado com o operador new. Os atributos de Veiculo, por exemplo, são o preço, a quilometragem e a situação; os métodos permitem consultar e alterar esse estado (Apêndice D).

5.2 Encapsulamento

Encapsular é esconder o estado interno do objeto e permitir o acesso apenas por métodos públicos (DEITEL; DEITEL, 2017). Na classe Veiculo, todos os atributos são private. Os métodos setPreco, setQuilometragem e setAnoFabricacao validam o valor recebido e lançam IllegalArgumentException quando a regra é violada, de modo que o objeto nunca fica em estado inválido. O identificador não possui método de alteração, pois é gerado pelo banco de dados. Esse princípio é o mesmo do exemplo da conta bancária estudado na disciplina, em que o saldo só muda por operações validadas (UNIFECAF, [s.d.b]).

5.3 Construtores e sobrecarga

O construtor define o estado inicial do objeto e tem o mesmo nome da classe (TURINI, 2014). A classe Veiculo possui dois construtores: um sem argumentos, exigido pela especificação de persistência (ECLIPSE FOUNDATION, 2022), e outro que recebe os dados principais. A existência de mais de um construtor com assinaturas diferentes caracteriza a sobrecarga. A classe Marca segue o mesmo padrão, com um construtor vazio e outro que recebe o nome.

5.4 Herança e composição

A herança permite que uma classe reaproveite atributos e métodos de outra, expressando uma relação “é um” (DEITEL; DEITEL, 2017). As entidades Marca, Modelo e Veiculo têm em comum o identificador, que fica na classe abstrata EntidadeBase, da qual as três herdam por meio de extends. Essa classe é anotada como superclasse mapeada, de modo que o atributo é persistido nas tabelas das subclasses (ECLIPSE FOUNDATION, 2022). Já a ligação entre Veiculo e Modelo, e entre Modelo e Marca, é de composição, uma relação “tem um”: o veículo possui um modelo, e não é um modelo.

5.5 Abstração e polimorfismo

A abstração consiste em modelar apenas o que é relevante para o problema. As classes do sistema não possuem, por exemplo, atributos de motor ou de dados fiscais, pois o objetivo é a gestão do estoque. O polimorfismo permite que um mesmo tipo de referência assuma formas diferentes (DEITEL; DEITEL, 2017). No sistema, os repositórios são interfaces que estendem JpaRepository; a camada de serviço depende apenas dessa interface, e o Spring Data fornece a implementação em tempo de execução (GIERKE et al., [s.d.]). O método toString, sobrescrito com @Override, é outro exemplo, pois a versão executada depende do tipo real do objeto.

5.6 Enumerações

Valores fixos, como o tipo e a situação do veículo, foram representados por enumerações (TipoVeiculo e StatusVeiculo). Isso impede valores digitados de formas diferentes, atende à resposta 8 do levantamento e permite usar esses campos como filtros de consulta.

5.7 Tratamento de exceções

Erros previsíveis são representados por exceções próprias: RecursoNaoEncontradoException, para registros inexistentes, e RegraDeNegocioException, para violações de regras. A classe TratadorDeErros, anotada com @RestControllerAdvice, captura essas exceções, bem como os erros de validação, e devolve uma resposta em JSON com o código HTTP adequado e uma mensagem legível (WEBB et al., 2024). Assim, a regra violada chega ao usuário sem expor detalhes internos da aplicação (Apêndice G).

6 ARQUITETURA EM CAMADAS E API REST

6.1 Camadas

O sistema separa as responsabilidades em camadas, no padrão MVC estudado na disciplina (UNIFECAF, [s.d.e], [s.d.f]), como mostra o Quadro 7. Cada camada depende apenas da camada logo abaixo, o que atende ao requisito de manutenibilidade (RNF04).

Quadro 7 – Camadas da aplicação

Camada (pacote)	**Classes**	**Função**
model	EntidadeBase, Marca, Modelo, Veiculo, TipoVeiculo, StatusVeiculo	Dados e regras do negócio; mapeados para tabelas do banco.
repository	MarcaRepository, ModeloRepository, VeiculoRepository	Acesso ao banco de dados por meio de JpaRepository (salvar, buscar, listar, excluir) e da consulta com filtros.
service	MarcaService, ModeloService, VeiculoService	Regras de negócio e validações; intermediária entre controlador e repositório.
controller	MarcaController, ModeloController, VeiculoController	Recebe as requisições HTTP e devolve as respostas em JSON.
exception	RecursoNaoEncontradoException, RegraDeNegocioException, TratadorDeErros	Exceções próprias e tratamento centralizado de erros.

Fonte: Elaborado pelo autor (2026).

6.2 API REST

A interface entre o servidor e a página web segue o estilo arquitetural REST, em que cada recurso é identificado por um endereço e manipulado por meio de métodos HTTP padronizados (FIELDING, 2000). O Quadro 8 apresenta as operações sobre veículos; marcas e modelos seguem o mesmo padrão nos caminhos /api/marcas e /api/modelos.

Quadro 8 – Operações da API REST sobre veículos

Operação	**Método e anotação**	**Caminho**	**Resposta**
Consultar / filtrar	GET (@GetMapping)	/api/veiculos	200 e lista em JSON
Buscar por identificador	GET (@GetMapping)	/api/veiculos/{id}	200 ou 404
Cadastrar	POST (@PostMapping)	/api/veiculos	201 ou 400
Atualizar	PUT (@PutMapping)	/api/veiculos/{id}	200, 400 ou 404
Remover	DELETE (@DeleteMapping)	/api/veiculos/{id}	204 ou 404

Fonte: Elaborado pelo autor (2026).

6.3 Consulta com filtros opcionais

A consulta de veículos aceita sete filtros opcionais: marca, modelo, preço mínimo, preço máximo, ano, situação e tipo. Em vez de criar um método para cada combinação, o repositório define uma única consulta em JPQL na qual cada parâmetro vazio é ignorado, por meio da condição “parâmetro é nulo ou campo igual ao parâmetro” (GIERKE et al., [s.d.]). O serviço converte textos vazios em valores nulos e verifica se o preço mínimo não excede o máximo antes de chamar o repositório (Apêndices E e F).

6.4 Interface web

A página web é um arquivo estático (index.html), servido pelo próprio Spring Boot, e utiliza a função fetch do JavaScript para chamar a API (MDN WEB DOCS, 2025). Ela possui abas para veículos, marcas e modelos, formulário de filtros, formulários de cadastro e edição, e exibe ao usuário as mensagens de sucesso ou de erro devolvidas pelo servidor (Apêndice H).

7 JUSTIFICATIVA DAS TECNOLOGIAS ESCOLHIDAS

O Quadro 9 relaciona as tecnologias utilizadas e o motivo da escolha de cada uma. As versões adotadas foram Java 17, Spring Boot 3.4.5, Apache Maven 3.8.7 e MySQL Community Server.

Quadro 9 – Tecnologias utilizadas e justificativas

Tecnologia	**Justificativa**
Java	Linguagem orientada a objetos, robusta e muito usada no mercado; permite aplicar diretamente classes, encapsulamento, herança e polimorfismo (DEITEL; DEITEL, 2017).
Spring Boot	Reduz a configuração inicial, traz servidor embutido e facilita a criação de APIs REST organizadas em camadas (WEBB et al., 2024).
Spring Data JPA e Hibernate	Mapeiam classes para tabelas e fornecem operações prontas de salvar, buscar, listar e excluir, sem necessidade de escrever SQL manualmente (GIERKE et al., [s.d.]; ECLIPSE FOUNDATION, 2022).
Bean Validation	Declara regras de validação por anotações (por exemplo, campo obrigatório), complementando as validações dos métodos de acesso.
MySQL	Banco de dados relacional gratuito e amplamente utilizado, compatível com o JPA e adequado a dados estruturados como os do estoque (ORACLE, [s.d.]).
Maven	Gerencia as dependências no arquivo pom.xml e automatiza a construção e a execução do projeto.
HTML, CSS e JavaScript	Permitem uma interface simples, sem necessidade de instalar outro servidor ou ferramenta de interface (MDN WEB DOCS, 2025).
Git e GitHub	Controle de versão e publicação do código-fonte para correção.

Fonte: Elaborado pelo autor (2026).

As configurações de conexão (endereço do banco, usuário e senha) ficam no arquivo application.properties e o driver do MySQL é incluído como dependência no pom.xml (Apêndices A e B). A senha que consta no apêndice é de uso exclusivo em teste, em ambiente local.

8 CONSIDERAÇÕES FINAIS

O sistema proposto responde ao problema de desorganização do estoque ao centralizar os dados, permitir consultas filtradas e manter as informações atualizadas. Do ponto de vista acadêmico, o trabalho integra o conteúdo da disciplina: fundamentos de orientação a objetos (Unidades I a III), acesso a banco de dados e MVC (Unidade IV) e Spring Boot com persistência e API REST (Unidade V).

A aplicação de encapsulamento, herança, composição, polimorfismo e tratamento de exceções resultou em um código organizado em camadas, com regras de validação concentradas e mensagens de erro claras. Como trabalhos futuros, sugerem-se a autenticação de usuários, a paginação das consultas, o registro de histórico de alterações de preço e a criação de testes automatizados. A execução do sistema e as evidências de funcionamento estão no relatório da parte prática, e o código-fonte completo está nos apêndices deste relatório e no repositório público do projeto.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 10520:2023**: informação e documentação: citações em documentos: apresentação. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 14724:2024**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2024.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6023:2018**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6024:2012**: informação e documentação: numeração progressiva das seções de um documento escrito: apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6027:2012**: informação e documentação: sumário: apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6028:2021**: informação e documentação: resumo, resenha e recensão: apresentação. Rio de Janeiro: ABNT, 2021.

DEITEL, Paul; DEITEL, Harvey. **Java**: como programar. 10. ed. São Paulo: Pearson, 2017.

ECLIPSE FOUNDATION. **Jakarta Persistence 3.1**. [S. l.]: Eclipse Foundation, 2022. Disponível em: https://jakarta.ee/specifications/persistence/3.1/. Acesso em: 3 nov. 2026.

FIELDING, Roy Thomas. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Informação e Ciência da Computação) – University of California, Irvine, 2000. Disponível em: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm. Acesso em: 3 nov. 2026.

GIERKE, Oliver *et al*. **Spring Data JPA reference documentation**: versão 4.1.2. [S. l.]: VMware, [s.d.]. Disponível em: https://docs.spring.io/spring-data/jpa/reference/. Acesso em: 3 nov. 2026.

MDN WEB DOCS. **Fetch API**. [S. l.]: Mozilla, 2025. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/API/Fetch_API. Acesso em: 3 nov. 2026.

ORACLE. **MySQL 8.0 reference manual**. [S. l.]: Oracle, [s.d.]. Disponível em: https://dev.mysql.com/doc/refman/8.0/en/. Acesso em: 3 nov. 2026.

TURINI, Rodrigo. **Desbravando Java e orientação a objetos**: um guia para o iniciante da linguagem. São Paulo: Casa do Código, 2014.

UNIFECAF. **Object Oriented Programming**: unidade I. [S. l.]: UNIFECAF, [s.d.b]. Material didático.

UNIFECAF. **Object Oriented Programming**: unidade II. [S. l.]: UNIFECAF, [s.d.c]. Material didático.

UNIFECAF. **Object Oriented Programming**: unidade III. [S. l.]: UNIFECAF, [s.d.d]. Material didático.

UNIFECAF. **Object Oriented Programming**: unidade IV. [S. l.]: UNIFECAF, [s.d.e]. Material didático.

UNIFECAF. **Object Oriented Programming**: unidade V. [S. l.]: UNIFECAF, [s.d.f]. Material didático.

UNIFECAF. **Trabalho – Object Oriented Programming**: sistema automotivo – gestão de estoque de veículos. [S. l.]: UNIFECAF, [s.d.a]. Enunciado do trabalho.

WEBB, Phillip *et al*. **Spring Boot reference documentation**: versão 3.2.x. [S. l.]: VMware, 2024. Disponível em: https://docs.spring.io/spring-boot/docs/3.2.x/reference/html. Acesso em: 3 nov. 2026.

APÊNDICE A – ARQUIVO DE DEPENDÊNCIAS DO PROJETO (POM.XML)

Arquivo de configuração do Apache Maven, com as dependências do Spring Boot (Web, Data JPA e Validation), do driver do MySQL e a versão do Java. As linhas longas foram quebradas apenas para a impressão.

Arquivo: pom.xml

```
 1  <?xml version="1.0" encoding="UTF-8"?>
 2  <project xmlns="http://maven.apache.org/POM/4.0.0"
 3           xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 4           xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
 5               https://maven.apache.org/xsd/maven-4.0.0.xsd">
 6      <modelVersion>4.0.0</modelVersion>
 7
 8      <parent>
 9          <groupId>org.springframework.boot</groupId>
10          <artifactId>spring-boot-starter-parent</artifactId>
11          <version>3.4.5</version>
12          <relativePath/>
13      </parent>
14
15      <groupId>br.com.estoque</groupId>
16      <artifactId>estoque-veiculos</artifactId>
17      <version>1.0.0</version>
18      <name>estoque-veiculos</name>
19      <description>Sistema Automotivo - Gestao de Estoque de Veiculos</description>
20
21      <properties>
22          <java.version>17</java.version>
23      </properties>
24
25      <dependencies>
26          <!-- API REST (controladores, JSON, servidor embutido) -->
27          <dependency>
28              <groupId>org.springframework.boot</groupId>
29              <artifactId>spring-boot-starter-web</artifactId>
30          </dependency>
31
32          <!-- Persistencia: JPA/Hibernate + Spring Data -->
33          <dependency>
34              <groupId>org.springframework.boot</groupId>
35              <artifactId>spring-boot-starter-data-jpa</artifactId>
36          </dependency>
37
38          <!-- Validacao dos dados recebidos (@NotBlank, @NotNull, @Min...) -->
39          <dependency>
40              <groupId>org.springframework.boot</groupId>
41              <artifactId>spring-boot-starter-validation</artifactId>
42          </dependency>
43
44          <!-- Driver de conexao com o MySQL -->
45          <dependency>
46              <groupId>com.mysql</groupId>
47              <artifactId>mysql-connector-j</artifactId>
48              <scope>runtime</scope>
49          </dependency>
50      </dependencies>
51
52      <build>
53          <plugins>
54              <plugin>
55                  <groupId>org.springframework.boot</groupId>
56                  <artifactId>spring-boot-maven-plugin</artifactId>
57              </plugin>
58          </plugins>
59      </build>
60  </project>
```

APÊNDICE B – CONFIGURAÇÃO DO BANCO DE DADOS E COMANDOS DE EXECUÇÃO

Arquivo application.properties, com a conexão ao MySQL, e comandos utilizados para preparar o banco de dados e executar o sistema. A senha é de teste, para uso local.

Arquivo: src/main/resources/application.properties

```
spring.application.name=estoque-veiculos

# Conexao com o MySQL (banco e usuario criados no roteiro)
spring.datasource.url=jdbc:mysql://localhost:3306/estoque_veiculos?useSSL=false&allowPub
    licKeyRetrieval=true
spring.datasource.username=estoque
spring.datasource.password=Estoque@2026

# Cria/atualiza as tabelas automaticamente a partir das classes
spring.jpa.hibernate.ddl-auto=update

# Mostra no terminal os comandos SQL enviados ao banco
spring.jpa.show-sql=true
spring.jpa.open-in-view=false
```

Comandos SQL (executados no cliente *mysql*, com usuário administrador):

```
DROP DATABASE IF EXISTS estoque_veiculos;
CREATE DATABASE estoque_veiculos CHARACTER SET utf8mb4;
CREATE USER IF NOT EXISTS 'estoque'@'localhost' IDENTIFIED BY 'Estoque@2026';
GRANT ALL PRIVILEGES ON estoque_veiculos.* TO 'estoque'@'localhost';
FLUSH PRIVILEGES;
SHOW DATABASES;
USE estoque_veiculos;
SHOW TABLES;
DESCRIBE veiculo;
SELECT v.id, m.nome AS marca, mo.nome AS modelo, v.tipo, v.ano_fabricacao, v.cor,
    v.preco, v.quilometragem, v.status
FROM veiculo v JOIN modelo mo ON v.modelo_id = mo.id JOIN marca m ON mo.marca_id = m.id
ORDER BY v.id;
```

Comandos de terminal (Linux):

```
java -version
mvn -version
sudo systemctl status mysql
mvn spring-boot:run
curl "http://localhost:8080/api/veiculos?marca=Honda&status=VENDIDO"
curl -X POST http://localhost:8080/api/veiculos -H "Content-Type: application/json" \
  -d '{"modelo":{"id":1},"tipo":"CARRO","anoFabricacao":2022,"cor":"Prata","preco":12000
      0.00,"quilometragem":15000,"status":"DISPONIVEL"}'
```

APÊNDICE C – CLASSE PRINCIPAL E ENUMERAÇÕES

Classe de inicialização da aplicação Spring Boot e enumerações TipoVeiculo e StatusVeiculo.

Arquivo: src/main/java/br/com/estoque/EstoqueApplication.java

```
 1  package br.com.estoque;
 2
 3  import org.springframework.boot.SpringApplication;
 4  import org.springframework.boot.autoconfigure.SpringBootApplication;
 5
 6  /**
 7   * Ponto de entrada da aplicacao. O Spring Boot inicia o servidor embutido
 8   * e descobre automaticamente as classes dos pacotes abaixo de br.com.estoque.
 9   */
10  @SpringBootApplication
11  public class EstoqueApplication {
12
13      public static void main(String[] args) {
14          SpringApplication.run(EstoqueApplication.class, args);
15      }
16  }
```

Arquivo: src/main/java/br/com/estoque/model/TipoVeiculo.java

```
1  package br.com.estoque.model;
2
3  /** Tipos de veiculo aceitos pelo sistema (usado tambem como filtro de busca). */
4  public enum TipoVeiculo {
5      CARRO,
6      MOTO,
7      CAMINHAO
8  }
```

Arquivo: src/main/java/br/com/estoque/model/StatusVeiculo.java

```
1  package br.com.estoque.model;
2
3  /** Situacao de disponibilidade de um veiculo no estoque. */
4  public enum StatusVeiculo {
5      DISPONIVEL,
6      RESERVADO,
7      VENDIDO
8  }
```

APÊNDICE D – CAMADA MODEL: ENTIDADES

Superclasse EntidadeBase e entidades Marca, Modelo e Veiculo, com encapsulamento, herança, composição e sobrecarga de construtores.

Arquivo: src/main/java/br/com/estoque/model/EntidadeBase.java

```
 1  package br.com.estoque.model;
 2
 3  import jakarta.persistence.GeneratedValue;
 4  import jakarta.persistence.GenerationType;
 5  import jakarta.persistence.Id;
 6  import jakarta.persistence.MappedSuperclass;
 7
 8  /**
 9   * Classe base (heranca): reune o que todas as entidades tem em comum, o identificador.
10   * Marca, Modelo e Veiculo herdam dela com "extends".
11   * O id e gerado pelo banco; por isso nao ha metodo setId (encapsulamento).
12   */
13  @MappedSuperclass
14  public abstract class EntidadeBase {
15
16      @Id
17      @GeneratedValue(strategy = GenerationType.IDENTITY)
18      private Long id;
19
20      public Long getId() {
21          return id;
22      }
23  }
```

Arquivo: src/main/java/br/com/estoque/model/Marca.java

```
 1  package br.com.estoque.model;
 2
 3  import jakarta.persistence.Column;
 4  import jakarta.persistence.Entity;
 5  import jakarta.persistence.Table;
 6  import jakarta.validation.constraints.NotBlank;
 7
 8  /** Fabricante do veiculo (por exemplo, Toyota). Uma marca possui varios modelos. */
 9  @Entity
10  @Table(name = "marca")
11  public class Marca extends EntidadeBase {
12
13      @NotBlank(message = "O nome da marca e obrigatorio")
14      @Column(nullable = false, unique = true, length = 60)
15      private String nome;
16
17      /** Construtor sem argumentos: exigido pelo JPA e pelo Jackson (leitura do JSON). */
18      public Marca() {
19      }
20
21      public Marca(String nome) {
22          this.nome = nome;
23      }
24
25      public String getNome() {
26          return nome;
27      }
28
29      public void setNome(String nome) {
30          this.nome = nome;
31      }
32
33      @Override
34      public String toString() {
35          return "Marca{id=" + getId() + ", nome='" + nome + "'}";
36      }
37  }
```

Arquivo: src/main/java/br/com/estoque/model/Modelo.java

```
 1  package br.com.estoque.model;
 2
 3  import jakarta.persistence.Column;
 4  import jakarta.persistence.Entity;
 5  import jakarta.persistence.JoinColumn;
 6  import jakarta.persistence.ManyToOne;
 7  import jakarta.persistence.Table;
 8  import jakarta.validation.constraints.NotBlank;
 9  import jakarta.validation.constraints.NotNull;
10
11  /** Modelo comercial (por exemplo, Corolla). Pertence a uma marca e possui varios
12      veiculos. */
13  @Entity
14  @Table(name = "modelo")
15  public class Modelo extends EntidadeBase {
16
17      @NotBlank(message = "O nome do modelo e obrigatorio")
18      @Column(nullable = false, length = 60)
19      private String nome;
20
21      // Composicao ("tem um"): cada modelo tem uma marca
22      @NotNull(message = "A marca e obrigatoria")
23      @ManyToOne(optional = false)
24      @JoinColumn(name = "marca_id", nullable = false)
25      private Marca marca;
26
27      public Modelo() {
28      }
29
30      public Modelo(String nome, Marca marca) {
31          this.nome = nome;
32          this.marca = marca;
33      }
34
35      public String getNome() {
36          return nome;
37      }
38
39      public void setNome(String nome) {
40          this.nome = nome;
41      }
42
43      public Marca getMarca() {
44          return marca;
45      }
46
47      public void setMarca(Marca marca) {
48          this.marca = marca;
49      }
50
51      @Override
52      public String toString() {
53          return "Modelo{id=" + getId() + ", nome='" + nome + "', marca=" + marca + "}";
54      }
55  }
```

Arquivo: src/main/java/br/com/estoque/model/Veiculo.java

```
 1  package br.com.estoque.model;
 2
 3  import jakarta.persistence.Column;
 4  import jakarta.persistence.Entity;
 5  import jakarta.persistence.EnumType;
 6  import jakarta.persistence.Enumerated;
 7  import jakarta.persistence.JoinColumn;
 8  import jakarta.persistence.ManyToOne;
 9  import jakarta.persistence.Table;
10  import jakarta.validation.constraints.Min;
11  import jakarta.validation.constraints.NotBlank;
12  import jakarta.validation.constraints.NotNull;
13  import jakarta.validation.constraints.Positive;
14
15  import java.time.Year;
16
17  /**
18   * Unidade fisica em estoque: entidade central do sistema.
19   * Os atributos sao privados (encapsulamento) e os setters validam as regras
20   * antes de alterar o estado do objeto.
21   */
22  @Entity
23  @Table(name = "veiculo")
24  public class Veiculo extends EntidadeBase {
25
26      @NotNull(message = "O modelo e obrigatorio")
27      @ManyToOne(optional = false)
28      @JoinColumn(name = "modelo_id", nullable = false)
29      private Modelo modelo;
30
31      @NotNull(message = "O tipo do veiculo e obrigatorio")
32      @Enumerated(EnumType.STRING)
33      @Column(nullable = false, length = 20)
34      private TipoVeiculo tipo;
35
36      @Min(value = 1900, message = "Informe o ano de fabricacao (a partir de 1900)")
37      @Column(nullable = false)
38      private int anoFabricacao;
39
40      @NotBlank(message = "A cor e obrigatoria")
41      @Column(nullable = false, length = 30)
42      private String cor;
43
44      @Positive(message = "O preco deve ser maior que zero")
45      @Column(nullable = false)
46      private double preco;
47
48      @Min(value = 0, message = "A quilometragem nao pode ser negativa")
49      @Column(nullable = false)
50      private int quilometragem;
51
52      @NotNull(message = "O status e obrigatorio")
53      @Enumerated(EnumType.STRING)
54      @Column(nullable = false, length = 20)
55      private StatusVeiculo status;
56
57      /** Construtor sem argumentos: exigido pelo JPA e pelo Jackson. */
58      public Veiculo() {
59      }
60
61      /** Construtor com os dados principais (sobrecarga de construtores). */
62      public Veiculo(Modelo modelo, TipoVeiculo tipo, int anoFabricacao, String cor,
63                     double preco, int quilometragem, StatusVeiculo status) {
64          this.modelo = modelo;
65          this.tipo = tipo;
66          this.cor = cor;
67          this.status = status;
68          setAnoFabricacao(anoFabricacao);
69          setPreco(preco);
70          setQuilometragem(quilometragem);
71      }
```

```
 72
 73      public Modelo getModelo() {
 74          return modelo;
 75      }
 76
 77      public void setModelo(Modelo modelo) {
 78          this.modelo = modelo;
 79      }
 80
 81      public TipoVeiculo getTipo() {
 82          return tipo;
 83      }
 84
 85      public void setTipo(TipoVeiculo tipo) {
 86          this.tipo = tipo;
 87      }
 88
 89      public int getAnoFabricacao() {
 90          return anoFabricacao;
 91      }
 92
 93      public void setAnoFabricacao(int anoFabricacao) {
 94          int limite = Year.now().getValue() + 1;
 95          if (anoFabricacao < 1900 || anoFabricacao > limite) {
 96              throw new IllegalArgumentException("Ano de fabricacao invalido: use um ano
 97                  entre 1900 e " + limite);
 98          }
 99          this.anoFabricacao = anoFabricacao;
100      }
101
102      public String getCor() {
103          return cor;
104      }
105
106      public void setCor(String cor) {
107          this.cor = cor;
108      }
109
110      public double getPreco() {
111          return preco;
112      }
113
114      public void setPreco(double preco) {
115          if (preco <= 0) {
116              throw new IllegalArgumentException("Preco invalido: o valor deve ser maior
117                  que zero");
118          }
119          this.preco = preco;
120      }
121
122      public int getQuilometragem() {
123          return quilometragem;
124      }
125
126      public void setQuilometragem(int quilometragem) {
127          if (quilometragem < 0) {
128              throw new IllegalArgumentException("Quilometragem invalida: nao pode ser
129                  negativa");
130          }
131          this.quilometragem = quilometragem;
132      }
133
134      public StatusVeiculo getStatus() {
135          return status;
136      }
137
138      public void setStatus(StatusVeiculo status) {
139          this.status = status;
140      }
141
142      @Override
143      public String toString() {
```

```
144          return "Veiculo{id=" + getId() + ", modelo=" + modelo + ", tipo=" + tipo
145                  + ", ano=" + anoFabricacao + ", cor='" + cor + "', preco=" + preco
146                  + ", km=" + quilometragem + ", status=" + status + "}";
147      }
148  }
```

APÊNDICE E – CAMADA REPOSITORY: ACESSO AO BANCO DE DADOS

Interfaces que estendem JpaRepository, incluindo a consulta JPQL com filtros opcionais de veículos.

Arquivo: src/main/java/br/com/estoque/repository/MarcaRepository.java

```
 1  package br.com.estoque.repository;
 2
 3  import br.com.estoque.model.Marca;
 4  import org.springframework.data.jpa.repository.JpaRepository;
 5
 6  /**
 7   * Interface: o Spring cria a implementacao em tempo de execucao (polimorfismo).
 8   * JpaRepository ja traz salvar, buscar, listar e excluir.
 9   */
10  public interface MarcaRepository extends JpaRepository<Marca, Long> {
11
12      boolean existsByNomeIgnoreCase(String nome);
13
14      boolean existsByNomeIgnoreCaseAndIdNot(String nome, Long id);
15  }
```

Arquivo: src/main/java/br/com/estoque/repository/ModeloRepository.java

```
1  package br.com.estoque.repository;
2
3  import br.com.estoque.model.Modelo;
4  import org.springframework.data.jpa.repository.JpaRepository;
5
6  public interface ModeloRepository extends JpaRepository<Modelo, Long> {
7  }
```

Arquivo: src/main/java/br/com/estoque/repository/VeiculoRepository.java

```
 1  package br.com.estoque.repository;
 2
 3  import br.com.estoque.model.StatusVeiculo;
 4  import br.com.estoque.model.TipoVeiculo;
 5  import br.com.estoque.model.Veiculo;
 6  import org.springframework.data.jpa.repository.JpaRepository;
 7  import org.springframework.data.jpa.repository.Query;
 8  import org.springframework.data.repository.query.Param;
 9
10  import java.util.List;
11
12  public interface VeiculoRepository extends JpaRepository<Veiculo, Long> {
13
14      /**
15       * Consulta com filtros opcionais: o filtro que vier vazio (null) e ignorado.
16       * Assim, uma unica consulta atende marca, modelo, preco, ano, status e tipo.
17       */
18      @Query("""
19              SELECT v FROM Veiculo v
20              WHERE (:marca IS NULL OR LOWER(v.modelo.marca.nome) = LOWER(:marca))
21                AND (:modelo IS NULL OR LOWER(v.modelo.nome) = LOWER(:modelo))
22                AND (:precoMin IS NULL OR v.preco >= :precoMin)
23                AND (:precoMax IS NULL OR v.preco <= :precoMax)
24                AND (:ano IS NULL OR v.anoFabricacao = :ano)
25                AND (:status IS NULL OR v.status = :status)
26                AND (:tipo IS NULL OR v.tipo = :tipo)
27              ORDER BY v.id
28              """)
29      List<Veiculo> filtrar(@Param("marca") String marca,
30                            @Param("modelo") String modelo,
31                            @Param("precoMin") Double precoMin,
32                            @Param("precoMax") Double precoMax,
33                            @Param("ano") Integer ano,
34                            @Param("status") StatusVeiculo status,
35                            @Param("tipo") TipoVeiculo tipo);
36  }
```

APÊNDICE F – CAMADA SERVICE: REGRAS DE NEGÓCIO

Classes de serviço, com as regras de negócio e validações.

Arquivo: src/main/java/br/com/estoque/service/MarcaService.java

```
 1  package br.com.estoque.service;
 2
 3  import br.com.estoque.exception.RecursoNaoEncontradoException;
 4  import br.com.estoque.exception.RegraDeNegocioException;
 5  import br.com.estoque.model.Marca;
 6  import br.com.estoque.repository.MarcaRepository;
 7  import org.springframework.data.domain.Sort;
 8  import org.springframework.stereotype.Service;
 9
10  import java.util.List;
11
12  /** Regras de negocio das marcas. */
13  @Service
14  public class MarcaService {
15
16      private final MarcaRepository repository;
17
18      public MarcaService(MarcaRepository repository) {
19          this.repository = repository;
20      }
21
22      public List<Marca> listar() {
23          return repository.findAll(Sort.by("nome"));
24      }
25
26      public Marca buscar(Long id) {
27          return repository.findById(id)
28                  .orElseThrow(() -> new RecursoNaoEncontradoException("Marca nao
29                      encontrada: id " + id));
30      }
31
32      public Marca criar(Marca dados) {
33          if (repository.existsByNomeIgnoreCase(dados.getNome())) {
34              throw new RegraDeNegocioException("Ja existe uma marca com o nome " +
35                  dados.getNome());
36          }
37          return repository.save(new Marca(dados.getNome()));
38      }
39
40      public Marca atualizar(Long id, Marca dados) {
41          Marca marca = buscar(id);
42          if (repository.existsByNomeIgnoreCaseAndIdNot(dados.getNome(), id)) {
43              throw new RegraDeNegocioException("Ja existe uma marca com o nome " +
44                  dados.getNome());
45          }
46          marca.setNome(dados.getNome());
47          return repository.save(marca);
48      }
49
50      public void remover(Long id) {
51          repository.delete(buscar(id));
52      }
53  }
```

Arquivo: src/main/java/br/com/estoque/service/ModeloService.java

```
 1  package br.com.estoque.service;
 2
 3  import br.com.estoque.exception.RecursoNaoEncontradoException;
 4  import br.com.estoque.exception.RegraDeNegocioException;
 5  import br.com.estoque.model.Marca;
 6  import br.com.estoque.model.Modelo;
 7  import br.com.estoque.repository.ModeloRepository;
 8  import org.springframework.data.domain.Sort;
 9  import org.springframework.stereotype.Service;
10
11  import java.util.List;
12
13  /** Regras de negocio dos modelos: todo modelo precisa estar ligado a uma marca
14      existente. */
15  @Service
16  public class ModeloService {
17
18      private final ModeloRepository repository;
19      private final MarcaService marcaService;
20
21      public ModeloService(ModeloRepository repository, MarcaService marcaService) {
22          this.repository = repository;
23          this.marcaService = marcaService;
24      }
25
26      public List<Modelo> listar() {
27          return repository.findAll(Sort.by("nome"));
28      }
29
30      public Modelo buscar(Long id) {
31          return repository.findById(id)
32                  .orElseThrow(() -> new RecursoNaoEncontradoException("Modelo nao
33                      encontrado: id " + id));
34      }
35
36      public Modelo criar(Modelo dados) {
37          return repository.save(new Modelo(dados.getNome(), buscarMarca(dados)));
38      }
39
40      public Modelo atualizar(Long id, Modelo dados) {
41          Modelo modelo = buscar(id);
42          modelo.setNome(dados.getNome());
43          modelo.setMarca(buscarMarca(dados));
44          return repository.save(modelo);
45      }
46
47      public void remover(Long id) {
48          repository.delete(buscar(id));
49      }
50
51      /** Converte a marca recebida no JSON ({"id":1}) na marca completa do banco. */
52      private Marca buscarMarca(Modelo dados) {
53          Long idMarca = dados.getMarca().getId();
54          if (idMarca == null) {
55              throw new RegraDeNegocioException("Informe o id da marca. Exemplo:
56                  \"marca\":{\"id\":1}");
57          }
58          return marcaService.buscar(idMarca);
59      }
60  }
```

Arquivo: src/main/java/br/com/estoque/service/VeiculoService.java

```
 1  package br.com.estoque.service;
 2
 3  import br.com.estoque.exception.RecursoNaoEncontradoException;
 4  import br.com.estoque.exception.RegraDeNegocioException;
 5  import br.com.estoque.model.Modelo;
 6  import br.com.estoque.model.StatusVeiculo;
 7  import br.com.estoque.model.TipoVeiculo;
 8  import br.com.estoque.model.Veiculo;
 9  import br.com.estoque.repository.VeiculoRepository;
10  import org.springframework.stereotype.Service;
11
12  import java.util.List;
13
14  /** Regras de negocio dos veiculos: intermediario entre o controlador e o repositorio.
15      */
16  @Service
17  public class VeiculoService {
18
19      private final VeiculoRepository repository;
20      private final ModeloService modeloService;
21
22      public VeiculoService(VeiculoRepository repository, ModeloService modeloService) {
23          this.repository = repository;
24          this.modeloService = modeloService;
25      }
26
27      public List<Veiculo> filtrar(String marca, String modelo, Double precoMin, Double
28          precoMax,
29                                   Integer ano, StatusVeiculo status, TipoVeiculo tipo) {
30          if (precoMin != null && precoMax != null && precoMin > precoMax) {
31              throw new RegraDeNegocioException("O preco minimo nao pode ser maior que o
32                  preco maximo");
33          }
34          return repository.filtrar(vazioParaNulo(marca), vazioParaNulo(modelo),
35                  precoMin, precoMax, ano, status, tipo);
36      }
37
38      public Veiculo buscar(Long id) {
39          return repository.findById(id)
40                  .orElseThrow(() -> new RecursoNaoEncontradoException("Veiculo nao
41                      encontrado: id " + id));
42      }
43
44      public Veiculo criar(Veiculo dados) {
45          Veiculo novo = new Veiculo(buscarModelo(dados), dados.getTipo(),
46              dados.getAnoFabricacao(),
47                  dados.getCor(), dados.getPreco(), dados.getQuilometragem(),
48                      dados.getStatus());
49          return repository.save(novo);
50      }
51
52      public Veiculo atualizar(Long id, Veiculo dados) {
53          Veiculo veiculo = buscar(id);
54          veiculo.setModelo(buscarModelo(dados));
55          veiculo.setTipo(dados.getTipo());
56          veiculo.setAnoFabricacao(dados.getAnoFabricacao());
57          veiculo.setCor(dados.getCor());
58          veiculo.setPreco(dados.getPreco());
59          veiculo.setQuilometragem(dados.getQuilometragem());
60          veiculo.setStatus(dados.getStatus());
61          return repository.save(veiculo);
62      }
63
64      public void remover(Long id) {
65          repository.delete(buscar(id));
66      }
67
68      /** Converte o modelo recebido no JSON ({"id":1}) no modelo completo do banco. */
69      private Modelo buscarModelo(Veiculo dados) {
70          Long idModelo = dados.getModelo().getId();
71          if (idModelo == null) {
```

```
72              throw new RegraDeNegocioException("Informe o id do modelo. Exemplo:
73                  \"modelo\":{\"id\":1}");
74          }
75          return modeloService.buscar(idModelo);
76      }
77
78      private String vazioParaNulo(String texto) {
79          return (texto == null || texto.isBlank()) ? null : texto.trim();
80      }
81  }
```

APÊNDICE G – CAMADA CONTROLLER E TRATAMENTO DE EXCEÇÕES

Controladores da API REST, exceções próprias e tratador global de erros.

Arquivo: src/main/java/br/com/estoque/controller/MarcaController.java

```
 1  package br.com.estoque.controller;
 2
 3  import br.com.estoque.model.Marca;
 4  import br.com.estoque.service.MarcaService;
 5  import jakarta.validation.Valid;
 6  import org.springframework.http.HttpStatus;
 7  import org.springframework.web.bind.annotation.DeleteMapping;
 8  import org.springframework.web.bind.annotation.GetMapping;
 9  import org.springframework.web.bind.annotation.PathVariable;
10  import org.springframework.web.bind.annotation.PostMapping;
11  import org.springframework.web.bind.annotation.PutMapping;
12  import org.springframework.web.bind.annotation.RequestBody;
13  import org.springframework.web.bind.annotation.RequestMapping;
14  import org.springframework.web.bind.annotation.ResponseStatus;
15  import org.springframework.web.bind.annotation.RestController;
16
17  import java.util.List;
18
19  /** Camada Controller: recebe as requisicoes HTTP de marcas e devolve JSON. */
20  @RestController
21  @RequestMapping("/api/marcas")
22  public class MarcaController {
23
24      private final MarcaService service;
25
26      public MarcaController(MarcaService service) {
27          this.service = service;
28      }
29
30      @GetMapping
31      public List<Marca> listar() {
32          return service.listar();
33      }
34
35      @GetMapping("/{id}")
36      public Marca buscar(@PathVariable Long id) {
37          return service.buscar(id);
38      }
39
40      @PostMapping
41      @ResponseStatus(HttpStatus.CREATED)
42      public Marca criar(@Valid @RequestBody Marca marca) {
43          return service.criar(marca);
44      }
45
46      @PutMapping("/{id}")
47      public Marca atualizar(@PathVariable Long id, @Valid @RequestBody Marca marca) {
48          return service.atualizar(id, marca);
49      }
50
51      @DeleteMapping("/{id}")
52      @ResponseStatus(HttpStatus.NO_CONTENT)
53      public void remover(@PathVariable Long id) {
54          service.remover(id);
55      }
56  }
```

Arquivo: src/main/java/br/com/estoque/controller/ModeloController.java

```
 1  package br.com.estoque.controller;
 2
 3  import br.com.estoque.model.Modelo;
 4  import br.com.estoque.service.ModeloService;
 5  import jakarta.validation.Valid;
 6  import org.springframework.http.HttpStatus;
 7  import org.springframework.web.bind.annotation.DeleteMapping;
 8  import org.springframework.web.bind.annotation.GetMapping;
 9  import org.springframework.web.bind.annotation.PathVariable;
10  import org.springframework.web.bind.annotation.PostMapping;
11  import org.springframework.web.bind.annotation.PutMapping;
12  import org.springframework.web.bind.annotation.RequestBody;
13  import org.springframework.web.bind.annotation.RequestMapping;
14  import org.springframework.web.bind.annotation.ResponseStatus;
15  import org.springframework.web.bind.annotation.RestController;
16
17  import java.util.List;
18
19  /** Camada Controller: recebe as requisicoes HTTP de modelos e devolve JSON. */
20  @RestController
21  @RequestMapping("/api/modelos")
22  public class ModeloController {
23
24      private final ModeloService service;
25
26      public ModeloController(ModeloService service) {
27          this.service = service;
28      }
29
30      @GetMapping
31      public List<Modelo> listar() {
32          return service.listar();
33      }
34
35      @GetMapping("/{id}")
36      public Modelo buscar(@PathVariable Long id) {
37          return service.buscar(id);
38      }
39
40      @PostMapping
41      @ResponseStatus(HttpStatus.CREATED)
42      public Modelo criar(@Valid @RequestBody Modelo modelo) {
43          return service.criar(modelo);
44      }
45
46      @PutMapping("/{id}")
47      public Modelo atualizar(@PathVariable Long id, @Valid @RequestBody Modelo modelo) {
48          return service.atualizar(id, modelo);
49      }
50
51      @DeleteMapping("/{id}")
52      @ResponseStatus(HttpStatus.NO_CONTENT)
53      public void remover(@PathVariable Long id) {
54          service.remover(id);
55      }
56  }
```

Arquivo: src/main/java/br/com/estoque/controller/VeiculoController.java

```
 1  package br.com.estoque.controller;
 2
 3  import br.com.estoque.model.StatusVeiculo;
 4  import br.com.estoque.model.TipoVeiculo;
 5  import br.com.estoque.model.Veiculo;
 6  import br.com.estoque.service.VeiculoService;
 7  import jakarta.validation.Valid;
 8  import org.springframework.http.HttpStatus;
 9  import org.springframework.web.bind.annotation.DeleteMapping;
10  import org.springframework.web.bind.annotation.GetMapping;
11  import org.springframework.web.bind.annotation.PathVariable;
12  import org.springframework.web.bind.annotation.PostMapping;
13  import org.springframework.web.bind.annotation.PutMapping;
14  import org.springframework.web.bind.annotation.RequestBody;
15  import org.springframework.web.bind.annotation.RequestMapping;
16  import org.springframework.web.bind.annotation.RequestParam;
17  import org.springframework.web.bind.annotation.ResponseStatus;
18  import org.springframework.web.bind.annotation.RestController;
19
20  import java.util.List;
21
22  /** Camada Controller: recebe as requisicoes HTTP de veiculos (CRUD e filtros) e
23      devolve JSON. */
24  @RestController
25  @RequestMapping("/api/veiculos")
26  public class VeiculoController {
27
28      private final VeiculoService service;
29
30      public VeiculoController(VeiculoService service) {
31          this.service = service;
32      }
33
34      /** Lista todos os veiculos ou aplica filtros opcionais, por exemplo
35          ?marca=Toyota&ano=2022. */
36      @GetMapping
37      public List<Veiculo> listar(@RequestParam(required = false) String marca,
38                                  @RequestParam(required = false) String modelo,
39                                  @RequestParam(required = false) Double precoMin,
40                                  @RequestParam(required = false) Double precoMax,
41                                  @RequestParam(required = false) Integer ano,
42                                  @RequestParam(required = false) StatusVeiculo status,
43                                  @RequestParam(required = false) TipoVeiculo tipo) {
44          return service.filtrar(marca, modelo, precoMin, precoMax, ano, status, tipo);
45      }
46
47      @GetMapping("/{id}")
48      public Veiculo buscar(@PathVariable Long id) {
49          return service.buscar(id);
50      }
51
52      @PostMapping
53      @ResponseStatus(HttpStatus.CREATED)
54      public Veiculo criar(@Valid @RequestBody Veiculo veiculo) {
55          return service.criar(veiculo);
56      }
57
58      @PutMapping("/{id}")
59      public Veiculo atualizar(@PathVariable Long id, @Valid @RequestBody Veiculo
60          veiculo) {
61          return service.atualizar(id, veiculo);
62      }
63
64      @DeleteMapping("/{id}")
65      @ResponseStatus(HttpStatus.NO_CONTENT)
66      public void remover(@PathVariable Long id) {
67          service.remover(id);
68      }
69  }
```

Arquivo: src/main/java/br/com/estoque/exception/RecursoNaoEncontradoException.java

```
 1  package br.com.estoque.exception;
 2
 3  /** Lancada quando um registro (marca, modelo ou veiculo) nao existe. Vira resposta
 4      404. */
 5  public class RecursoNaoEncontradoException extends RuntimeException {
 6
 7      public RecursoNaoEncontradoException(String mensagem) {
 8          super(mensagem);
 9      }
10  }
```

Arquivo: src/main/java/br/com/estoque/exception/RegraDeNegocioException.java

```
 1  package br.com.estoque.exception;
 2
 3  /** Lancada quando uma regra do sistema e violada (por exemplo, marca repetida). Vira
 4      resposta 400. */
 5  public class RegraDeNegocioException extends RuntimeException {
 6
 7      public RegraDeNegocioException(String mensagem) {
 8          super(mensagem);
 9      }
10  }
```

Arquivo: src/main/java/br/com/estoque/exception/TratadorDeErros.java

```
 1  package br.com.estoque.exception;
 2
 3  import com.fasterxml.jackson.databind.exc.InvalidFormatException;
 4  import com.fasterxml.jackson.databind.JsonMappingException;
 5  import org.springframework.dao.DataIntegrityViolationException;
 6  import org.springframework.http.HttpStatus;
 7  import org.springframework.http.ResponseEntity;
 8  import org.springframework.http.converter.HttpMessageNotReadableException;
 9  import org.springframework.web.bind.MethodArgumentNotValidException;
10  import org.springframework.web.bind.annotation.ExceptionHandler;
11  import org.springframework.web.bind.annotation.RestControllerAdvice;
12  import org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;
13
14  import java.util.LinkedHashMap;
15  import java.util.Map;
16
17  /**
18   * Transforma erros em respostas JSON claras, no formato {"erro": "mensagem"},
19   * em vez de mostrar a pilha de erros do Java ao usuario da API.
20   */
21  @RestControllerAdvice
22  public class TratadorDeErros {
23
24      @ExceptionHandler(RecursoNaoEncontradoException.class)
25      public ResponseEntity<Map<String, Object>>
26          naoEncontrado(RecursoNaoEncontradoException ex) {
27          return resposta(HttpStatus.NOT_FOUND, ex.getMessage());
28      }
29
30      @ExceptionHandler(RegraDeNegocioException.class)
31      public ResponseEntity<Map<String, Object>> regraDeNegocio(RegraDeNegocioException
32          ex) {
33          return resposta(HttpStatus.BAD_REQUEST, ex.getMessage());
34      }
35
36      /** Falha de validacao das anotacoes (@NotBlank, @NotNull, @Min...). */
37      @ExceptionHandler(MethodArgumentNotValidException.class)
38      public ResponseEntity<Map<String, Object>>
39          validacao(MethodArgumentNotValidException ex) {
40          Map<String, String> campos = new LinkedHashMap<>();
41          ex.getBindingResult().getFieldErrors()
42                  .forEach(erro -> campos.putIfAbsent(erro.getField(),
43                      erro.getDefaultMessage()));
44          Map<String, Object> corpo = new LinkedHashMap<>();
45          corpo.put("erro", "Dados invalidos");
46          corpo.put("campos", campos);
47          return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(corpo);
48      }
49
50      /** JSON mal formado ou valor recusado pelos setters (por exemplo, preco negativo).
51          */
52      @ExceptionHandler(HttpMessageNotReadableException.class)
53      public ResponseEntity<Map<String, Object>>
54          corpoInvalido(HttpMessageNotReadableException ex) {
55          Throwable causa = ex.getMostSpecificCause();
56          String mensagem;
57          if (causa instanceof IllegalArgumentException) {
58              mensagem = causa.getMessage();
59          } else if (causa instanceof InvalidFormatException formato) {
60              var caminho = formato.getPath();
61              String campo = caminho.isEmpty() ? "?" : caminho.get(caminho.size() -
62                  1).getFieldName();
63              mensagem = "Valor invalido para o campo '" + campo + "': " +
64                  formato.getValue();
65          } else if (causa instanceof JsonMappingException) {
66              mensagem = "Dados invalidos no corpo da requisicao";
67          } else {
68              mensagem = "Corpo da requisicao invalido (confira o JSON enviado)";
69          }
70          return resposta(HttpStatus.BAD_REQUEST, mensagem);
71      }
```

```
72
73      /** Valor incorreto em um filtro da URL (por exemplo, status=XYZ). */
74      @ExceptionHandler(MethodArgumentTypeMismatchException.class)
75      public ResponseEntity<Map<String, Object>>
76          parametroInvalido(MethodArgumentTypeMismatchException ex) {
77          return resposta(HttpStatus.BAD_REQUEST,
78                  "Valor invalido para o parametro '" + ex.getName() + "': " +
79                      ex.getValue());
80      }
81
82      /** Por exemplo, tentar remover uma marca que ainda possui modelos. */
83      @ExceptionHandler(DataIntegrityViolationException.class)
84      public ResponseEntity<Map<String, Object>>
85          integridade(DataIntegrityViolationException ex) {
86          return resposta(HttpStatus.CONFLICT,
87                  "Operacao nao permitida: o registro esta em uso ou viola uma regra do
88                      banco de dados");
89      }
90
91      private ResponseEntity<Map<String, Object>> resposta(HttpStatus status, String
92          mensagem) {
93          Map<String, Object> corpo = new LinkedHashMap<>();
94          corpo.put("erro", mensagem);
95          return ResponseEntity.status(status).body(corpo);
96      }
97  }
```

APÊNDICE H – INTERFACE WEB (INDEX.HTML)

Página web estática, com HTML, CSS e JavaScript, que consome a API REST.

Arquivo: src/main/resources/static/index.html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Gestão de Estoque de Veículos</title>
<style>
  :root {
    --azul: #1f3a5f; --azul-claro: #e8eef6; --borda: #d3dbe6; --fundo: #f4f6f9;
    --verde: #1b7f4b; --amarelo: #a86a00; --vermelho: #b3261e; --texto: #1c2430;
  }
  * { box-sizing: border-box; }
  body { margin: 0; font-family: system-ui, -apple-system, "Segoe UI", Roboto, Arial,
      sans-serif;
         background: var(--fundo); color: var(--texto); }
  header { background: var(--azul); color: #fff; padding: 16px 24px; display: flex;
      flex-wrap: wrap;
           align-items: center; justify-content: space-between; gap: 12px; }
  header h1 { font-size: 20px; margin: 0; }
  header small { display: block; opacity: .8; font-weight: 400; margin-top: 2px; }
  main { max-width: 1180px; margin: 0 auto; padding: 20px 16px 60px; }
  nav { display: flex; gap: 8px; margin-bottom: 16px; }
  nav button { border: 1px solid var(--borda); background: #fff; padding: 9px 18px;
      border-radius: 8px;
               cursor: pointer; font-size: 15px; color: var(--azul); }
  nav button.ativo { background: var(--azul); color: #fff; border-color: var(--azul); }
  section.painel { display: none; }
  section.painel.ativo { display: block; }
  .cartao { background: #fff; border: 1px solid var(--borda); border-radius: 10px;
      padding: 16px; margin-bottom: 16px; }
  .cartao h2 { font-size: 16px; margin: 0 0 12px; color: var(--azul); }
  .grade { display: grid; grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));
      gap: 12px; align-items: end; }
  label { display: block; font-size: 12px; font-weight: 600; color: #455068;
      margin-bottom: 4px; }
  input, select { width: 100%; padding: 8px 10px; border: 1px solid var(--borda);
      border-radius: 6px;
                  font-size: 14px; background: #fff; color: var(--texto); }
  input:focus, select:focus { outline: 2px solid #8fb0dd; border-color: #8fb0dd; }
  .acoes { display: flex; gap: 8px; margin-top: 12px; flex-wrap: wrap; }
  button.btn { padding: 9px 16px; border-radius: 6px; border: 1px solid var(--azul);
      background: var(--azul);
               color: #fff; cursor: pointer; font-size: 14px; }
  button.btn.sec { background: #fff; color: var(--azul); }
  button.btn.perigo { background: #fff; color: var(--vermelho); border-color:
      var(--vermelho); }
  button.btn.peq { padding: 4px 10px; font-size: 12px; }
  table { width: 100%; border-collapse: collapse; font-size: 14px; }
  th { text-align: left; background: var(--azul-claro); color: var(--azul); padding:
      9px 10px; font-size: 12px;
       text-transform: uppercase; letter-spacing: .03em; }
  td { padding: 9px 10px; border-top: 1px solid var(--borda); }
  td.num, th.num { text-align: right; }
  tr:hover td { background: #fafbfd; }
  .selo { display: inline-block; padding: 2px 10px; border-radius: 12px; font-size:
      12px; font-weight: 600; }
  .DISPONIVEL { background: #dff3e8; color: var(--verde); }
  .RESERVADO { background: #fdeccc; color: var(--amarelo); }
  .VENDIDO { background: #f6d9d6; color: var(--vermelho); }
  .contagem { font-size: 13px; color: #55627a; margin: 0 0 10px; }
  .vazio { text-align: center; color: #66738b; padding: 24px; }
  #aviso { position: fixed; right: 16px; bottom: 16px; max-width: 420px; padding: 12px
      16px; border-radius: 8px;
           color: #fff; font-size: 14px; display: none; box-shadow: 0 4px 16px
```

```
                rgba(0,0,0,.25); z-index: 10; }
  #aviso.ok { background: var(--verde); display: block; }
  #aviso.erro { background: var(--vermelho); display: block; }
  .oculto { display: none; }
  .tabela-rolagem { overflow-x: auto; }
</style>
</head>
<body>
<header>
  <h1>Sistema Automotivo – Gestão de Estoque de Veículos
    <small>Java · Spring Boot · MySQL · API REST</small></h1>
  <button class="btn sec" id="btnExemplo" title="Cadastra marcas, modelos e veículos de
      exemplo">Carregar dados de exemplo</button>
</header>

<main>
  <nav>
    <button class="ativo" data-aba="veiculos">Veículos</button>
    <button data-aba="marcas">Marcas</button>
    <button data-aba="modelos">Modelos</button>
  </nav>

  <!-- ==================== VEÍCULOS ==================== -->
  <section class="painel ativo" id="aba-veiculos">
    <div class="cartao">
      <h2>Consulta e filtros</h2>
      <form id="formFiltro" class="grade">
        <div><label>Marca</label><select id="fMarca"></select></div>
        <div><label>Modelo</label><input id="fModelo" placeholder="Ex.: Corolla"></div>
        <div><label>Preço mínimo (R$)</label><input id="fPrecoMin" type="number"
            min="0" step="0.01"></div>
        <div><label>Preço máximo (R$)</label><input id="fPrecoMax" type="number"
            min="0" step="0.01"></div>
        <div><label>Ano</label><input id="fAno" type="number" min="1900"
            max="2100"></div>
        <div><label>Situação</label>
          <select id="fStatus"><option
              value="">Todas</option><option>DISPONIVEL</option><option>RESERVADO</optio
              n><option>VENDIDO</option></select></div>
        <div><label>Tipo</label>
          <select id="fTipo"><option
              value="">Todos</option><option>CARRO</option><option>MOTO</option><option>
              CAMINHAO</option></select></div>
        <div class="acoes" style="margin:0">
          <button class="btn" type="submit">Buscar</button>
          <button class="btn sec" type="button" id="btnLimpar">Limpar</button>
        </div>
      </form>
    </div>

    <div class="cartao">
      <h2 id="tituloFormVeiculo">Cadastrar veículo</h2>
      <form id="formVeiculo" class="grade">
        <input type="hidden" id="vId">
        <div><label>Modelo (marca)</label><select id="vModelo" required></select></div>
        <div><label>Tipo</label>
          <select id="vTipo"><option>CARRO</option><option>MOTO</option><option>CAMINHAO
              </option></select></div>
        <div><label>Ano de fabricação</label><input id="vAno" type="number"
            required></div>
        <div><label>Cor</label><input id="vCor" required></div>
        <div><label>Preço (R$)</label><input id="vPreco" type="number" step="0.01"
            required></div>
        <div><label>Quilometragem</label><input id="vKm" type="number" required></div>
        <div><label>Situação</label>
          <select id="vStatus"><option>DISPONIVEL</option><option>RESERVADO</option><opt
              ion>VENDIDO</option></select></div>
        <div class="acoes" style="margin:0">
          <button class="btn" type="submit" id="btnSalvarVeiculo">Cadastrar</button>
          <button class="btn sec oculto" type="button" id="btnCancelarEdicao">Cancelar
              edição</button>
        </div>
```

```
      </form>
    </div>

    <div class="cartao">
      <h2>Veículos em estoque</h2>
      <p class="contagem" id="contagem"></p>
      <div class="tabela-rolagem">
        <table>
          <thead><tr><th>ID</th><th>Marca</th><th>Modelo</th><th>Tipo</th><th
              class="num">Ano</th><th>Cor</th>
            <th class="num">Preço</th><th
                class="num">Km</th><th>Situação</th><th>Ações</th></tr></thead>
          <tbody id="tbVeiculos"></tbody>
        </table>
      </div>
    </div>
  </section>

  <!-- ===================== MARCAS ===================== -->
  <section class="painel" id="aba-marcas">
    <div class="cartao">
      <h2 id="tituloFormMarca">Cadastrar marca</h2>
      <form id="formMarca" class="grade">
        <input type="hidden" id="mId">
        <div><label>Nome da marca</label><input id="mNome" required placeholder="Ex.:
            Toyota"></div>
        <div class="acoes" style="margin:0">
          <button class="btn" type="submit" id="btnSalvarMarca">Cadastrar</button>
          <button class="btn sec oculto" type="button" id="btnCancelarMarca">Cancelar
              edição</button>
        </div>
      </form>
    </div>
    <div class="cartao">
      <h2>Marcas cadastradas</h2>
      <table><thead><tr><th>ID</th><th>Nome</th><th>Ações</th></tr></thead><tbody
          id="tbMarcas"></tbody></table>
    </div>
  </section>

  <!-- ===================== MODELOS ===================== -->
  <section class="painel" id="aba-modelos">
    <div class="cartao">
      <h2 id="tituloFormModelo">Cadastrar modelo</h2>
      <form id="formModelo" class="grade">
        <input type="hidden" id="oId">
        <div><label>Nome do modelo</label><input id="oNome" required placeholder="Ex.:
            Corolla"></div>
        <div><label>Marca</label><select id="oMarca" required></select></div>
        <div class="acoes" style="margin:0">
          <button class="btn" type="submit" id="btnSalvarModelo">Cadastrar</button>
          <button class="btn sec oculto" type="button" id="btnCancelarModelo">Cancelar
              edição</button>
        </div>
      </form>
    </div>
    <div class="cartao">
      <h2>Modelos cadastrados</h2>
      <table><thead><tr><th>ID</th><th>Modelo</th><th>Marca</th><th>Ações</th></tr></the
          ad><tbody id="tbModelos"></tbody></table>
    </div>
  </section>
</main>

<div id="aviso" role="status"></div>

<script>
const API = '/api';
let marcas = [], modelos = [], veiculos = [];
const $ = id => document.getElementById(id);
const esc = s => String(s ?? '').replace(/[&<>"']/g, c =>
    ({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;","'":'&#39;'}[c]));
```

```
const moeda = v => Number(v).toLocaleString('pt-BR', { style: 'currency', currency:
    'BRL' });
const numero = v => Number(v).toLocaleString('pt-BR');

let temporizador;
function aviso(texto, tipo = 'ok') {
  const el = $('aviso');
  el.textContent = texto; el.className = tipo;
  clearTimeout(temporizador);
  temporizador = setTimeout(() => { el.className = ''; }, tipo === 'ok' ? 3500 : 7000);
}

async function api(caminho, opcoes = {}) {
  let r;
  try {
    r = await fetch(API + caminho, { headers: { 'Content-Type': 'application/json' },
        ...opcoes });
  } catch (e) {
    throw new Error('Não foi possível conectar ao sistema. Ele está ligado?');
  }
  if (r.status === 204) return null;
  let dados = null;
  try { dados = await r.json(); } catch (e) { /* resposta sem corpo */ }
  if (!r.ok) {
    let msg = (dados && dados.erro) ? dados.erro : 'Erro ' + r.status;
    if (dados && dados.campos) msg += ': ' + Object.values(dados.campos).join('; ');
    throw new Error(msg);
  }
  return dados;
}

async function executar(acao, mensagemOk) {
  try { await acao(); if (mensagemOk) aviso(mensagemOk); }
  catch (e) { aviso(e.message, 'erro'); }
}

/* ---------------- abas ---------------- */
document.querySelectorAll('nav button').forEach(b => b.addEventListener('click', () => {
  document.querySelectorAll('nav button').forEach(x => x.classList.toggle('ativo', x
      === b));
  document.querySelectorAll('.painel').forEach(p => p.classList.toggle('ativo', p.id
      === 'aba-' + b.dataset.aba));
}));

/* ---------------- carregamento ---------------- */
async function carregarBase() {
  marcas = await api('/marcas');
  modelos = await api('/modelos');
  desenharMarcas(); desenharModelos(); preencherSelects();
}

function consultaFiltro() {
  const p = new URLSearchParams();
  const add = (k, v) => { if (v !== '' && v != null) p.append(k, v); };
  add('marca', $('fMarca').value); add('modelo', $('fModelo').value.trim());
  add('precoMin', $('fPrecoMin').value); add('precoMax', $('fPrecoMax').value);
  add('ano', $('fAno').value); add('status', $('fStatus').value); add('tipo',
      $('fTipo').value);
  const s = p.toString();
  return '/veiculos' + (s ? '?' + s : '');
}

async function carregarVeiculos() {
  veiculos = await api(consultaFiltro());
  desenharVeiculos();
}

async function recarregarTudo() {
  await carregarBase();
  await carregarVeiculos();
}

```

```
/* ---------------- desenho das tabelas ---------------- */
function preencherSelects() {
  const marcaAtual = $('fMarca').value;
  $('fMarca').innerHTML = '<option value="">Todas</option>' + marcas.map(m =>
      `<option>${esc(m.nome)}</option>`).join('');
  $('fMarca').value = marcaAtual;
  const sel = $('vModelo').value;
  $('vModelo').innerHTML = modelos.length
    ? modelos.map(o => `<option value="${o.id}">${esc(o.marca.nome)} –
        ${esc(o.nome)}</option>`).join('')
    : '<option value="">Cadastre um modelo primeiro</option>';
  if (sel) $('vModelo').value = sel;
  const selMarca = $('oMarca').value;
  $('oMarca').innerHTML = marcas.length
    ? marcas.map(m => `<option value="${m.id}">${esc(m.nome)}</option>`).join('')
    : '<option value="">Cadastre uma marca primeiro</option>';
  if (selMarca) $('oMarca').value = selMarca;
}

function desenharVeiculos() {
  $('contagem').textContent = veiculos.length === 1 ? '1 veículo encontrado' :
      veiculos.length + ' veículos encontrados';
  $('tbVeiculos').innerHTML = veiculos.length ? veiculos.map(v => `
    <tr>
      <td>${v.id}</td><td>${esc(v.modelo.marca.nome)}</td><td>${esc(v.modelo.nome)}</td>
          <td>${esc(v.tipo)}</td>
      <td class="num">${v.anoFabricacao}</td><td>${esc(v.cor)}</td>
      <td class="num">${moeda(v.preco)}</td><td
          class="num">${numero(v.quilometragem)}</td>
      <td><span class="selo ${esc(v.status)}">${esc(v.status)}</span></td>
      <td><button class="btn sec peq" onclick="editarVeiculo(${v.id})">Editar</button>
          <button class="btn perigo peq"
              onclick="removerVeiculo(${v.id})">Remover</button></td>
    </tr>`).join('') : '<tr><td colspan="10" class="vazio">Nenhum veículo
        encontrado.</td></tr>';
}

function desenharMarcas() {
  $('tbMarcas').innerHTML = marcas.length ? marcas.map(m => `
    <tr><td>${m.id}</td><td>${esc(m.nome)}</td>
    <td><button class="btn sec peq" onclick="editarMarca(${m.id})">Editar</button>
        <button class="btn perigo peq"
            onclick="removerMarca(${m.id})">Remover</button></td></tr>`).join('')
    : '<tr><td colspan="3" class="vazio">Nenhuma marca cadastrada.</td></tr>';
}

function desenharModelos() {
  $('tbModelos').innerHTML = modelos.length ? modelos.map(o => `
    <tr><td>${o.id}</td><td>${esc(o.nome)}</td><td>${esc(o.marca.nome)}</td>
    <td><button class="btn sec peq" onclick="editarModelo(${o.id})">Editar</button>
        <button class="btn perigo peq"
            onclick="removerModelo(${o.id})">Remover</button></td></tr>`).join('')
    : '<tr><td colspan="4" class="vazio">Nenhum modelo cadastrado.</td></tr>';
}

/* ---------------- veículos: filtro, cadastro, edição, remoção ---------------- */
$('formFiltro').addEventListener('submit', e => { e.preventDefault();
    executar(carregarVeiculos); });
$('btnLimpar').addEventListener('click', () => { $('formFiltro').reset();
    executar(carregarVeiculos); });

function dadosVeiculo() {
  return {
    modelo: { id: Number($('vModelo').value) },
    tipo: $('vTipo').value,
    anoFabricacao: Number($('vAno').value),
    cor: $('vCor').value.trim(),
    preco: Number($('vPreco').value),
    quilometragem: Number($('vKm').value),
    status: $('vStatus').value
  };
}
```

```
353
354 function sairEdicaoVeiculo() {
355   $('formVeiculo').reset(); $('vId').value = '';
356   $('tituloFormVeiculo').textContent = 'Cadastrar veículo';
357   $('btnSalvarVeiculo').textContent = 'Cadastrar';
358   $('btnCancelarEdicao').classList.add('oculto');
359 }
360 $('btnCancelarEdicao').addEventListener('click', sairEdicaoVeiculo);
361
362 $('formVeiculo').addEventListener('submit', e => {
363   e.preventDefault();
364   const id = $('vId').value;
365   executar(async () => {
366     if (id) await api('/veiculos/' + id, { method: 'PUT', body:
367         JSON.stringify(dadosVeiculo()) });
368     else await api('/veiculos', { method: 'POST', body: JSON.stringify(dadosVeiculo())
369         });
370     sairEdicaoVeiculo();
371     await carregarVeiculos();
372   }, id ? 'Veículo atualizado com sucesso.' : 'Veículo cadastrado com sucesso.');
373 });
374
375 window.editarVeiculo = id => {
376   const v = veiculos.find(x => x.id === id); if (!v) return;
377   $('vId').value = v.id; $('vModelo').value = v.modelo.id; $('vTipo').value = v.tipo;
378   $('vAno').value = v.anoFabricacao; $('vCor').value = v.cor; $('vPreco').value =
379       v.preco;
380   $('vKm').value = v.quilometragem; $('vStatus').value = v.status;
381   $('tituloFormVeiculo').textContent = 'Editar veículo ' + v.id;
382   $('btnSalvarVeiculo').textContent = 'Salvar alterações';
383   $('btnCancelarEdicao').classList.remove('oculto');
384   $('formVeiculo').scrollIntoView({ behavior: 'smooth', block: 'center' });
385 };
386
387 window.removerVeiculo = id => {
388   if (!confirm('Remover o veículo ' + id + ' do estoque?')) return;
389   executar(async () => { await api('/veiculos/' + id, { method: 'DELETE' }); await
390       carregarVeiculos(); },
391            'Veículo removido do estoque.');
392 };
393
394 /* ---------------- marcas ---------------- */
395 function sairEdicaoMarca() {
396   $('formMarca').reset(); $('mId').value = '';
397   $('tituloFormMarca').textContent = 'Cadastrar marca'; $('btnSalvarMarca').textContent
398       = 'Cadastrar';
399   $('btnCancelarMarca').classList.add('oculto');
400 }
401 $('btnCancelarMarca').addEventListener('click', sairEdicaoMarca);
402 $('formMarca').addEventListener('submit', e => {
403   e.preventDefault();
404   const id = $('mId').value, corpo = JSON.stringify({ nome: $('mNome').value.trim() });
405   executar(async () => {
406     if (id) await api('/marcas/' + id, { method: 'PUT', body: corpo });
407     else await api('/marcas', { method: 'POST', body: corpo });
408     sairEdicaoMarca(); await recarregarTudo();
409   }, id ? 'Marca atualizada.' : 'Marca cadastrada.');
410 });
411 window.editarMarca = id => {
412   const m = marcas.find(x => x.id === id); if (!m) return;
413   $('mId').value = m.id; $('mNome').value = m.nome;
414   $('tituloFormMarca').textContent = 'Editar marca ' + m.id;
415       $('btnSalvarMarca').textContent = 'Salvar alterações';
416   $('btnCancelarMarca').classList.remove('oculto');
417 };
418 window.removerMarca = id => {
419   if (!confirm('Remover a marca ' + id + '?')) return;
420   executar(async () => { await api('/marcas/' + id, { method: 'DELETE' }); await
421       recarregarTudo(); }, 'Marca removida.');
422 };
423
424 /* ---------------- modelos ---------------- */
```

```
425 function sairEdicaoModelo() {
426   $('formModelo').reset(); $('oId').value = '';
427   $('tituloFormModelo').textContent = 'Cadastrar modelo';
428       $('btnSalvarModelo').textContent = 'Cadastrar';
429   $('btnCancelarModelo').classList.add('oculto');
430 }
431 $('btnCancelarModelo').addEventListener('click', sairEdicaoModelo);
432 $('formModelo').addEventListener('submit', e => {
433   e.preventDefault();
434   const id = $('oId').value;
435   const corpo = JSON.stringify({ nome: $('oNome').value.trim(), marca: { id:
436       Number($('oMarca').value) } });
437   executar(async () => {
438     if (id) await api('/modelos/' + id, { method: 'PUT', body: corpo });
439     else await api('/modelos', { method: 'POST', body: corpo });
440     sairEdicaoModelo(); await recarregarTudo();
441   }, id ? 'Modelo atualizado.' : 'Modelo cadastrado.');
442 });
443 window.editarModelo = id => {
444   const o = modelos.find(x => x.id === id); if (!o) return;
445   $('oId').value = o.id; $('oNome').value = o.nome; $('oMarca').value = o.marca.id;
446   $('tituloFormModelo').textContent = 'Editar modelo ' + o.id;
447       $('btnSalvarModelo').textContent = 'Salvar alterações';
448   $('btnCancelarModelo').classList.remove('oculto');
449 };
450 window.removerModelo = id => {
451   if (!confirm('Remover o modelo ' + id + '?')) return;
452   executar(async () => { await api('/modelos/' + id, { method: 'DELETE' }); await
453       recarregarTudo(); }, 'Modelo removido.');
454 };
455
456 /* ---------------- dados de exemplo ---------------- */
457 $('btnExemplo').addEventListener('click', () => executar(async () => {
458   const exMarcas = ['Toyota', 'Chevrolet', 'Honda', 'Volkswagen'];
459   const exModelos = [['Corolla', 'Toyota'], ['Onix', 'Chevrolet'], ['CG 160', 'Honda'],
460       ['Civic', 'Honda'],
461                     ['Constellation', 'Volkswagen']];
462   const exVeiculos = [
463     ['Corolla', 'CARRO', 2022, 'Prata', 120000, 15000, 'DISPONIVEL'],
464     ['Onix', 'CARRO', 2020, 'Branco', 65000, 48000, 'DISPONIVEL'],
465     ['Onix', 'CARRO', 2023, 'Preto', 85000, 9000, 'RESERVADO'],
466     ['CG 160', 'MOTO', 2023, 'Vermelha', 16000, 5000, 'DISPONIVEL'],
467     ['Civic', 'CARRO', 2021, 'Cinza', 105000, 30000, 'VENDIDO'],
468     ['Constellation', 'CAMINHAO', 2019, 'Branco', 310000, 120000, 'DISPONIVEL']
469   ];
470   await carregarBase();
471   for (const nome of exMarcas)
472     if (!marcas.some(m => m.nome === nome)) await api('/marcas', { method: 'POST',
473         body: JSON.stringify({ nome }) });
474   await carregarBase();
475   for (const [nome, marca] of exModelos)
476     if (!modelos.some(o => o.nome === nome && o.marca.nome === marca))
477       await api('/modelos', { method: 'POST', body: JSON.stringify({ nome, marca: { id:
478           marcas.find(m => m.nome === marca).id } }) });
479   await carregarBase();
480   const existentes = await api('/veiculos');
481   if (existentes.length === 0) {
482     for (const [modelo, tipo, ano, cor, preco, km, status] of exVeiculos)
483       await api('/veiculos', { method: 'POST', body: JSON.stringify({
484         modelo: { id: modelos.find(o => o.nome === modelo).id }, tipo, anoFabricacao:
485             ano, cor, preco, quilometragem: km, status }) });
486   }
487   await recarregarTudo();
488 }, 'Dados de exemplo carregados.'));
489
490 /* ---------------- início ---------------- */
491 executar(recarregarTudo);
492 </script>
493 </body>
494 </html>
```

APÊNDICE I – MODELO FÍSICO DO BANCO DE DADOS (SCRIPT SQL)

Script de criação do banco de dados e das tabelas marca, modelo e veiculo, com chaves, restrições e índices, compatível com MySQL 8.0.16 ou superior. Corresponde ao esquema gerado pela aplicação e está no arquivo modelo_fisico.sql do repositório.

Arquivo: modelo_fisico.sql

```
-- ======================================================================
-- Sistema Automotivo - Gestao de Estoque de Veiculos
-- Modelo fisico do banco de dados (MySQL 8.0.16 ou superior)
--
-- Equivale ao esquema que a aplicacao cria automaticamente
-- (spring.jpa.hibernate.ddl-auto=update), com as restricoes explicitas.
-- ATENCAO: o script apaga as tabelas existentes e seus dados.
-- Uso: sudo mysql < database/modelo_fisico.sql
-- ======================================================================

CREATE DATABASE IF NOT EXISTS estoque_veiculos CHARACTER SET utf8mb4;
USE estoque_veiculos;

DROP TABLE IF EXISTS veiculo;
DROP TABLE IF EXISTS modelo;
DROP TABLE IF EXISTS marca;

-- Marca: fabricante do veiculo (1 marca possui N modelos)
CREATE TABLE marca (
    id   BIGINT      NOT NULL AUTO_INCREMENT,
    nome VARCHAR(60) NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT uk_marca_nome UNIQUE (nome)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4;

-- Modelo: modelo comercial (1 modelo pertence a 1 marca e possui N veiculos)
CREATE TABLE modelo (
    id       BIGINT      NOT NULL AUTO_INCREMENT,
    nome     VARCHAR(60) NOT NULL,
    marca_id BIGINT      NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT fk_modelo_marca FOREIGN KEY (marca_id) REFERENCES marca (id)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4;

-- Veiculo: unidade em estoque (entidade central)
CREATE TABLE veiculo (
    id             BIGINT      NOT NULL AUTO_INCREMENT,
    modelo_id      BIGINT      NOT NULL,
    tipo           ENUM('CARRO', 'MOTO', 'CAMINHAO')              NOT NULL,
    ano_fabricacao INT         NOT NULL,
    cor            VARCHAR(30) NOT NULL,
    preco          DOUBLE      NOT NULL,
    quilometragem  INT         NOT NULL,
    status         ENUM('DISPONIVEL', 'RESERVADO', 'VENDIDO')     NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT fk_veiculo_modelo FOREIGN KEY (modelo_id) REFERENCES modelo (id),
    CONSTRAINT ck_veiculo_ano   CHECK (ano_fabricacao >= 1900),
    CONSTRAINT ck_veiculo_preco CHECK (preco > 0),
    CONSTRAINT ck_veiculo_km    CHECK (quilometragem >= 0)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4;

-- Indices de apoio aos filtros de consulta
CREATE INDEX idx_veiculo_status ON veiculo (status);
CREATE INDEX idx_veiculo_ano    ON veiculo (ano_fabricacao);
CREATE INDEX idx_veiculo_preco  ON veiculo (preco);
```